平衡树

一、引言：为什么需要平衡树?

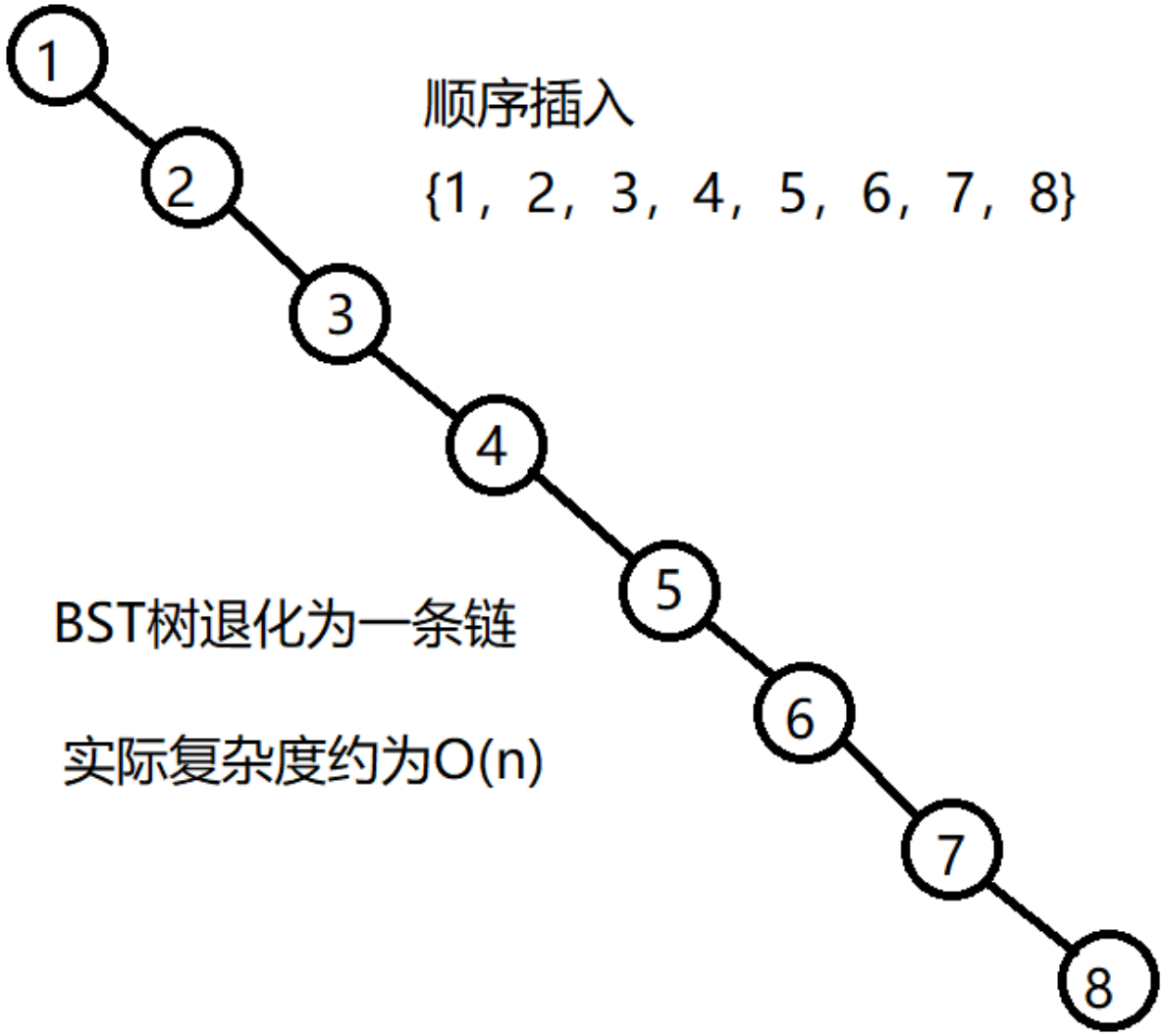
1.1 二叉搜索树（BST）的回顾与局限

核心思想回顾:

- 左子树所有节点值 < 根节点值 < 右子树所有节点值
- 中序遍历得到有序序列

BST的理想与现实:

- **理想情况**：每次插入/查询时间复杂度 O(log n)
- **最坏情况**：退化成链，时间复杂度 O(n)



1.2 平衡树的诞生

核心目标：通过**调整树的结构**，保证树的高度始终维持在 O(log n) 级别。即从任意节点出发，到其他任意节点的距离都是O(log n)。

两种主流思想：

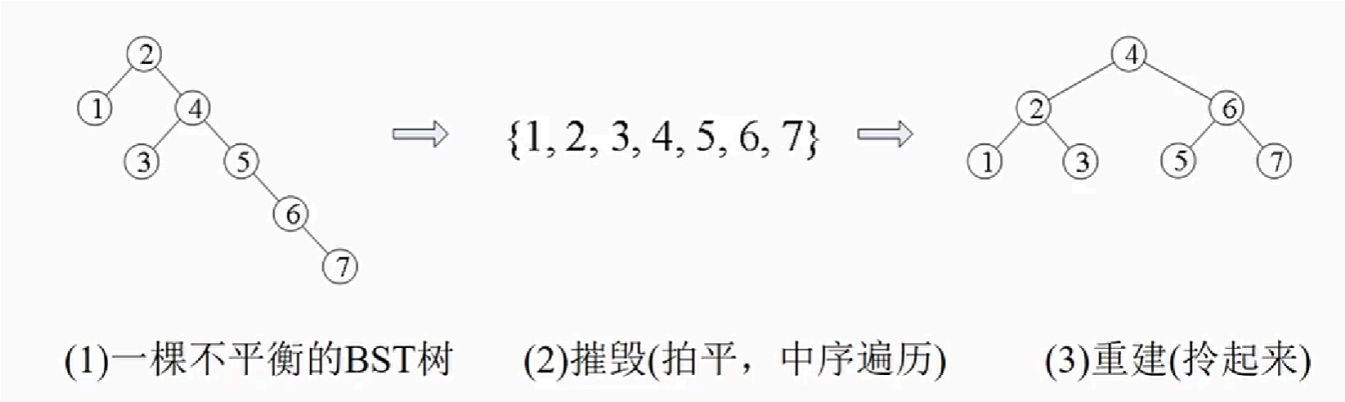
1. **随机化平衡**：Treap（树堆）
2. **旋转调整**：AVL、红黑树、Splay（伸展树）

二、替罪羊树基础概念

2.1 什么是替罪羊树?

替罪羊树（Scapegoat Tree）是一种**无需旋转操作**的平衡二叉搜索树，它用一种非常简单而暴力的方法维护二叉树的平衡：如果发现树上有一颗子树不平衡了，就摧毁它（先用中序遍历出子树上所有元素，再删除整颗子树），然后以中间元素为根，重新建一颗平衡的子树。“替罪羊树”的名称与它的原理有关。如果插入或删除树上的一个节点后，导致这个点所在的子树不再平衡，称这颗子树的根节点为替罪羊，因为这颗“倒霉”的子树即将被摧毁和重建。

把摧毁子树形象的形容为“拍平”，把用中间元素为根重建平衡树形容为“拎起来”。插入和删除节点后，如果发现BST不平衡，就进行拍平和拎起来两步操作。



2.2 核心思想

1. **惰性平衡**：只在需要时才进行重构，不像AVL树那样每次插入都检查平衡
2. **静态重构**：将不平衡子树转为有序数组，再重新构建为完全平衡的BST
3. **内存复用**：支持惰性删除，节点空间可重复利用

2.3 不平衡率

如果每加入或删除一个节点，就立刻“摧毁、重建”整棵树，代价太大，只要维护一颗“差不多平衡”的二叉树就可以了。那么，需要引入一个评估参数来对二叉树的不平衡性进行评估，这个参数就是不平衡率α。

一颗以u为根的子树，如果它的左子树或右子树占比大于α，就认为不平衡，α的取值范围为0.5<=α<=1,α越大，树越不平衡。例如α=0.5时，二叉树是绝对平衡的，表现为一颗满二叉树，每个节点的左右子树的数量几乎相同；α=1时，二叉树完全不平衡，退化成一条链。

不平衡率	0.5	0.55	0.6	0.65	0.7	0.75	0.8	0.85	0.9	0.95	1
重建次数	1003045	42642	24178	16960	13985	14196	7740	6640	5160	3380	0
树的深度	10	11	12	13	15	18	19	22	27	39	1000

一般把不平衡率设置为0.7左右。

三、数据结构与内存管理

3.1 节点定义

```
// 节点结构定义
struct Node{
    int ls,rs;   // 左右儿子指针
    int val;     // 结点存储的数字
    int tot;     // 子树占用的空间总量（包括已删除节点）
    int size;    // 子树上实际存储的数字数量
    int del;     // 删除标记：1=有效，0=已删除
}t[N];
```

3.2 内存管理

```
int order[N],cnt; //order[]记录拍扁后的结果，即那些存有数字的结点。cnt是数量
int tree_stack[N],top = 0;  //用一个栈来回收和分配可用的结点
int root = 0;               //根结点，注意重建过程中根结点会变化

// 初始化节点
void Initnode(int u){
    t[u].ls = t[u].rs = 0;
    t[u].size = t[u].tot = t[u].del = 1;
}

void Update(int u){
    t[u].size = t[t[u].ls].size + t[t[u].rs].size + (t[u].del ? 1 : 0);
    t[u].tot = t[t[u].ls].tot + t[t[u].rs].tot + 1;
}

// 预分配所有节点
for(int i = N-1; i >= 1; i--)
    tree_stack[++top] = i;
```

内存管理优势:

1. 避免频繁new/delete操作
2. 提高缓存命中率
3. 防止内存碎片

四、核心操作：拍平与重建

4.1 拍平操作（Flatten）

```
void inorder(int u){        //中序遍历，“拍平”摧毁这棵子树
    if(!u)  return;         //已经到达叶子，退出
    inorder(t[u].ls);       //先遍历左子树
    if(t[u].del)  order[++cnt] = u;      //如果该结点存有数字，读取它
    else tree_stack[++top] = u; //回收该结点，等待重新分配使用
    inorder(t[u].rs);       //再遍历右子树
}
```

4.2 重建操作（Rebuild）

```
void build(int l,int r,int &u){           //把拍平的子树拎起来，重建
    int mid = (l + r) >> 1;               //新的根设为中点，使重构出的树尽量平衡
    u = order[mid];
    if(l == r){Initnode(u); return;}      //如果是叶子，重置后返回
    if(l < mid)  build(l,mid - 1,t[u].ls);//重构左子树
    if(l == mid) t[u].ls = 0;             //注意这里，不要漏了
    build(mid + 1,r,t[u].rs);             //重构右子树
    Update(u);                            //更新
}
```

算法分析：

- 每次选择中点作为根，构建出的树是**完全平衡**的
- 递归深度为O(log n)
- 重建后树高最小，查询效率最高

4.3 平衡检查与触发重建

```
const double alpha = 0.75;  // 常用不平衡率

// 不平衡判断：基于α
bool notbalance(int u){
    // 当较大子树的size >= 当前节点size的α倍，则不平衡
    if((double)t[u].size * alpha <= (double)max(t[t[u].ls].size, t[t[u].rs].size))
        return true;
    return false;
}

// 重建入口
void rebuild(int &u){                        //重建。注意是&u
    cnt = 0;
    inorder(u);                              //先拍平摧毁
    if(cnt)   build(1,cnt,u);                //再拎起，重建树
    else u = 0;                          //特判该子树为空的情况
}
```

α说明：

- 常用值：α = 0.7 或 0.75
- α越小，树越平衡，但重建频率越高
- α越大，重建越少，但树可能更不平衡

五、例题 P3369 【模板】普通平衡树

题目描述

您需要动态地维护一个可重集合 M，并且提供以下操作：

1. 向 M 中插入一个数 x。
2. 从 M 中删除一个数 x（若有多个相同的数，应只删除一个）。
3. 查询 M 中有多少个数比 x 小，并且将得到的答案加一。

4. 查询如果将 M 从小到大排列后，排名位于第 x 位的数。
5. 查询 M 中 x 的前驱（前驱定义为小于 x，且最大的数）。
6. 查询 M 中 x 的后继（后继定义为大于 x，且最小的数）。

对于操作 $3, 5, 6$，**不保证**当前可重集中存在数 x。

对于操作 $5, 6$，保证答案一定存在。

输入格式

第一行为 n，表示操作的个数，下面 n 行每行有两个数 opt 和 x，opt 表示操作的序号（$1 \leq \text{opt} \leq 6$）。

输出格式

对于操作 $3, 4, 5, 6$ 每行输出一个数，表示对应答案。

输入输出样例 #1

输入 #1

```
10
1 106465
4 1
1 317721
1 460929
1 644985
1 84185
1 89851
6 81968
1 492737
5 493598
```

输出 #1

```
106465
84185
492737
```

说明/提示

【数据范围】

对于 100% 的数据，$1 \le n \le 10^5$，$|x| \le 10^7$。

在使用替罪羊树之前，先给出STL代码作为参考

```cpp
#include<bits/stdc++.h>
using namespace std;
vector<int> v;
int main(){
    int n;
    cin>>n;
    while(n--){
        int opt,x;
        cin>>opt>>x;
        switch (opt){
            case 1:
                v.insert(lower_bound(v.begin(),v.end(),x),x);
                break;
            case 2:
                v.erase(lower_bound(v.begin(),v.end(),x));
                break;
            case 3:
                cout<<lower_bound(v.begin(),v.end(),x)-v.begin()+1<<endl;
                break;
            case 4:
                cout<<v[x-1]<<endl;
                break;
            case 5:
                cout<<v[lower_bound(v.begin(),v.end(),x)-v.begin()-1]<<endl;
                break;
            case 6:
                cout<<v[upper_bound(v.begin(),v.end(),x)-v.begin()]<<endl;
                break;
        }
    }
    return 0;
}

```